

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ



BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	: Kim Ngọc Bách
Họ và tên sinh viên	: Đỗ Bình Minh
Ngày sinh	: 24/12/2005
Số điện thoại	: 0329279863
Mã sinh viên	: B23DCCN542
Lớp	: D23CQCN10-B
Nhóm lớp	: 11

Hà Nội – 2026

Mục lục

1. Mục tiêu công việc trong tuần.....	3
2. Các công việc đã thực hiện trong tuần	3
2.1. Trang chi tiết CV	3
2.1.1. Xử lý bên phía Frontend.....	3
2.1.2. Xử lý bên phía backend.....	5
2.1.3. Kết quả	8
3. Kết quả đạt được	8
4. Khó khăn, vướng mắc	9
5. Hướng giải quyết.....	9
6. Kế hoạch tuần tiếp theo	10

1. Mục tiêu công việc trong tuần

Trong tuần này, mục tiêu chính là xây dựng và hoàn thiện chức năng **xem chi tiết CV ứng viên** dành cho tài khoản công ty/nhà tuyển dụng. Chức năng này cho phép nhà tuyển dụng xem thông tin chi tiết của một CV đã ứng tuyển vào công việc thuộc công ty mình quản lý.

Cụ thể, công việc cần thực hiện bao gồm:

- Xây dựng trang chi tiết CV ở phía Frontend bằng Next.js.
- Lấy dữ liệu chi tiết CV dựa trên slug được truyền từ URL.
- Gửi cookie/token đăng nhập từ phía Frontend lên Backend để xác thực tài khoản công ty.
- Xây dựng API Backend lấy chi tiết CV theo id.
- Kiểm tra quyền truy cập, đảm bảo công ty chỉ được xem CV ứng tuyển vào công việc của chính công ty đó.
- Trả dữ liệu chi tiết CV và thông tin công việc liên quan về phía Frontend.

2. Các công việc đã thực hiện trong tuần

2.1. Trang chi tiết CV

2.1.1. Xử lý bên phía Frontend

```
export default async function CompanyManageCVDetailPage({ params }: {
  params: {
    slug: string
  }
}) {
  const { slug } = await params;
  const headersList = await headers(); // lấy headers từ request
  const cookie = headersList.get('cookie'); // lấy cookie từ headers

  const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/cv/detail/${slug}`, {
    method: "GET",
    headers: {
      cookie: cookie || ""
    },
    cache: "no-store"
  });
  const data = await res.json();
  let cvDetail: any = null;
  let jobDetail: any = null;
  if(data.code == "success") {
    cvDetail = data.cvDetail;
    jobDetail = data.jobDetail;
  }

  const position = positionList.find(pos => pos.value == jobDetail.position);
  const workingForm = workingFormList.find(work => work.value == jobDetail.workingForm);
```

Đoạn code này là **Server Component trong Next.js** dùng để lấy chi tiết CV theo slug.

```
export default async function CompanyManageCVDetailPage({ params })
```

Hàm page chạy bất đồng bộ để lấy dữ liệu trước khi render giao diện.

```
const { slug } = await params;
```

Lấy slug từ URL. Ví dụ URL dạng:

```
/company/manage-cv/detail/cv-123
```

thì slug = "cv-123".

```
const headersList = await headers();
```

```
const cookie = headersList.get("cookie");
```

Lấy cookie từ request hiện tại. Cookie này thường dùng để gửi kèm token đăng nhập lên API.

```
const res = await
```

```
fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/cv/detail/${slug}`, {
```

```
  method: "GET",
```

```
  headers: {
```

```
    cookie: cookie || ""
```

```
  },
```

```
  cache: "no-store"
```

```
});
```

Gọi API lấy chi tiết CV theo slug.

cache: "no-store" nghĩa là **không dùng cache**, mỗi lần vào trang sẽ gọi API mới.

```
const data = await res.json();
```

```
let cvDetail: any = null;
```

```
let jobDetail: any = null;
```

Chuyển dữ liệu API trả về sang JSON, sau đó tạo 2 biến để lưu:

```
cvDetail
```

```
jobDetail
```

```
if(data.code === "success") {
```

```
  cvDetail = data.cvDetail;
```

```
  jobDetail = data.jobDetail;
```

```
}
```

Nếu API trả về thành công thì gán dữ liệu CV và công việc vào biến.

```
const position = positionList.find(pos => pos.value === jobDetail.position);
```

```
const workingForm = workingFormList.find(work => work.value === jobDetail.workingForm);
```

Tìm thông tin hiển thị tương ứng với position và workingForm.

Ví dụ:

jobDetail.position = "frontend"

thì nó tìm trong positionList phần tử có:

value == "frontend"

để lấy label hiển thị như "Frontend Developer".

2.1.2. Xử lý bên phía backend

```
router.get(  
  "/cv/detail/:id",  
  authMiddleware.verifyTokenCompany,  
  companyController.detailCV  
)
```

Khai báo route dùng để cho công ty đã đăng nhập xem chi tiết một CV theo id.

```
export const detailCV = async (req: AccountRequest, res: Response) => {  
  try {  
    const cvId = req.params.id;  
    const companyId = req.account.id;  
  
    const infoCV = await CV.findOne({  
      _id: cvId  
    })  
  
    if(!infoCV) {  
      res.json({  
        code: "error",  
        message: "Dữ liệu không hợp lệ!"  
      })  
      return;  
    }  
  
    const infoJob = await Job.findOne({  
      _id: infoCV.jobId,  
      companyId: companyId  
    });  
  
    if(!infoJob) {  
      res.json({  
        code: "error",  
        message: "Dữ liệu không hợp lệ!"  
      })  
      return;  
    }  
  
    const dataFinalCV = {  
      fullName: infoCV.fullName,  
      email: infoCV.email,  
      phone: infoCV.phone,  
      fileCV: infoCV.fileCV,  
    };  
  
    const dataFinalJob = {  
      id: infoJob.id,  
      title: infoJob.title,  
      salaryMin: infoJob.salaryMin,  
      salaryMax: infoJob.salaryMax,  
      position: infoJob.position,  
      workingForm: infoJob.workingForm,  
      technologies: infoJob.technologies,  
    };  
  
    await CV.updateOne({  
      _id: cvId  
    }, {  
      viewed: true  
    })  
  
    res.json({  
      code: "success",  
      message: "Thành công!",  
      cvDetail: dataFinalCV,  
      jobDetail: dataFinalJob  
    })  
  } catch (error) {  
    console.log(error);  
    res.json({  
      code: "error",  
      message: "Dữ liệu không hợp lệ!"  
    })  
  }  
}
```

Đây là hàm **controller detailCV** bên backend, dùng để xử lý API xem chi tiết CV.

```
const cvId = req.params.id;  
const companyId = req.account.id;
```

Lấy:

```
cvId    = id CV trên URL  
companyId = id công ty đang đăng nhập
```

Ví dụ route:

```
/cv/detail/:id
```

thì :id chính là req.params.id.

```
const infoCV = await CV.findOne({  
  _id: cvId  
});
```

Tìm CV trong database theo cvId.

Nếu không tìm thấy:

```
if(!infoCV) {  
  res.json({  
    code: "error",  
    message: "Dữ liệu không hợp lệ!"  
  });  
  return;  
}
```

Trả về lỗi và dừng hàm.

```
const infoJob = await Job.findOne({  
  _id: infoCV.jobId,  
  companyId: companyId  
});
```

Tìm công việc mà CV này đã ứng tuyển.

Đồng thời kiểm tra công việc đó có thuộc về công ty đang đăng nhập không.

Ý nghĩa quan trọng: **công ty chỉ được xem CV ứng tuyển vào job của chính công ty đó.**

```
const dataFinalCV = {  
  fullName: infoCV.fullName,  
  email: infoCV.email,  
  phone: infoCV.phone,
```

```
fileCV: infoCV.fileCV,  
};
```

Tạo object chứa thông tin CV cần trả về frontend.

Không trả toàn bộ document từ database, chỉ trả các trường cần thiết.

```
const dataFinalJob = {  
  id: infoJob.id,  
  title: infoJob.title,  
  salaryMin: infoJob.salaryMin,  
  salaryMax: infoJob.salaryMax,  
  position: infoJob.position,  
  workingForm: infoJob.workingForm,  
  technologies: infoJob.technologies,  
};
```

Tạo object chứa thông tin job liên quan đến CV.

```
await CV.updateOne({  
  _id: cvId  
}, {  
  viewed: true  
});
```

Sau khi công ty xem CV, hệ thống cập nhật:

viewed: true

Tức là đánh dấu CV này là **đã xem**.

```
res.json({  
  code: "success",  
  message: "Thành công!",  
  cvDetail: dataFinalCV,  
  jobDetail: dataFinalJob  
});
```

Nếu mọi thứ hợp lệ, API trả về cho frontend:

```
cvDetail  
jobDetail
```

Đúng với đoạn frontend trước đó:

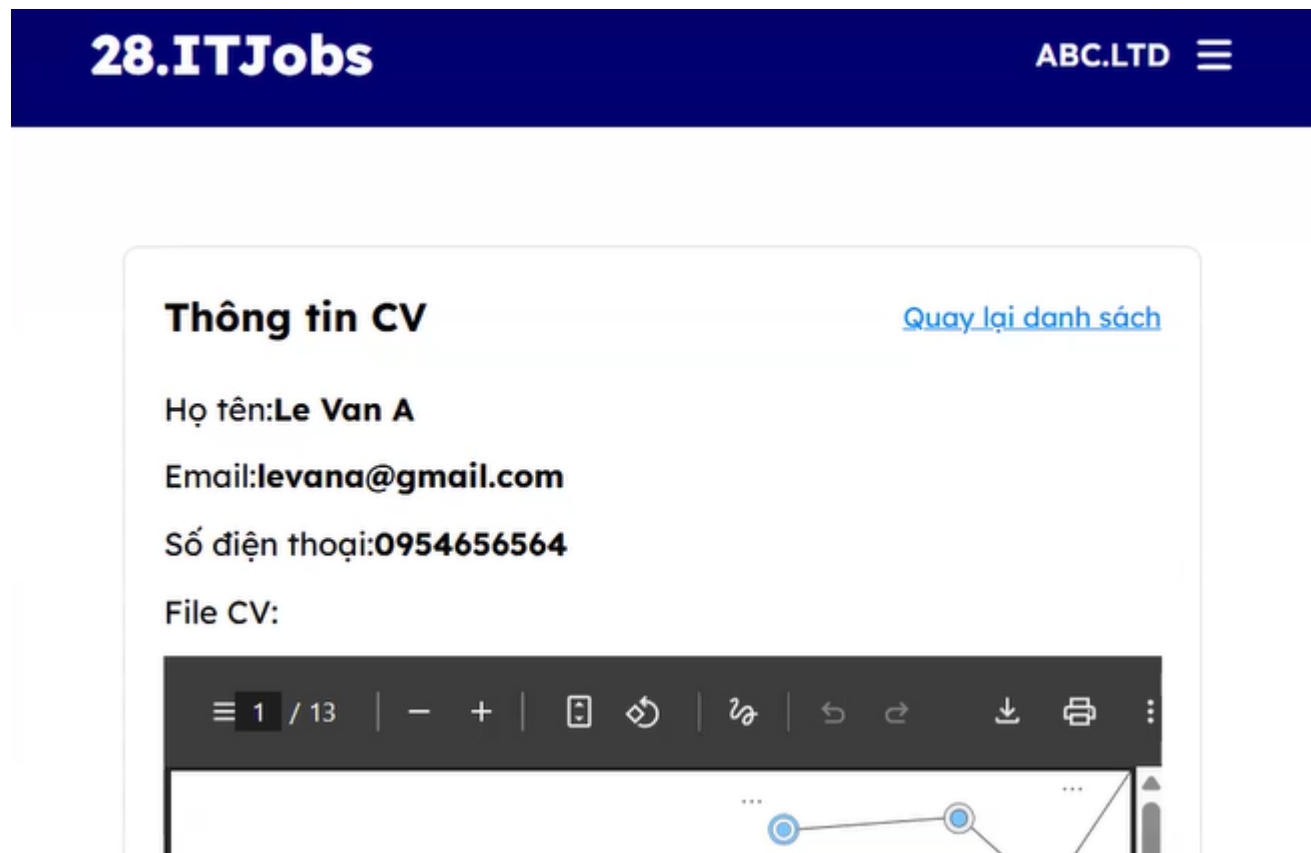
```
cvDetail = data.cvDetail;  
jobDetail = data.jobDetail;
```

Phần này:

```
catch (error) {
  console.log(error);
  res.json({
    code: "error",
    message: "Dữ liệu không hợp lệ!"
  });
}
```

Dùng để bắt lỗi bất ngờ, ví dụ cvId sai định dạng hoặc lỗi database.

2.1.3. Kết quả



Khi nhà tuyển dụng click vào cv thì thông tin chi tiết cv đó sẽ hiện ra

3. Kết quả đạt được

Sau khi thực hiện, chức năng **trang chi tiết CV** đã hoàn thành các yêu cầu cơ bản.

Về phía Frontend, hệ thống đã lấy được slug từ URL, gửi request đến API Backend và nhận dữ liệu chi tiết CV trả về. Dữ liệu sau khi nhận được được tách thành hai phần chính là thông tin CV và thông tin công việc liên quan. Ngoài ra, các trường như vị trí công việc và hình thức làm việc cũng được xử lý để hiển thị đúng nội dung cho người dùng.

Về phía Backend, API xem chi tiết CV đã được xây dựng thành công. Backend có kiểm tra token của tài khoản công ty thông qua middleware `verifyTokenCompany`. Sau đó, hệ thống tìm

CV theo id, kiểm tra công việc liên quan có thuộc về công ty đang đăng nhập hay không, rồi mới trả dữ liệu về cho Frontend.

Chức năng cũng đã hỗ trợ cập nhật trạng thái viewed: true sau khi nhà tuyển dụng xem chi tiết CV. Điều này giúp hệ thống có thể phân biệt được CV nào đã được công ty xem và CV nào chưa được xem.

Kết quả cuối cùng là khi nhà tuyển dụng click vào một CV trong danh sách, hệ thống sẽ chuyển sang trang chi tiết và hiển thị được thông tin của ứng viên cùng với thông tin công việc mà ứng viên đã ứng tuyển.

4. Khó khăn, vướng mắc

Trong quá trình thực hiện chức năng này, em gặp một số khó khăn như sau:

Thứ nhất, khi xử lý ở phía Frontend, cần lấy cookie từ request hiện tại trong Server Component của Next.js. Nếu không gửi cookie lên Backend thì API sẽ không xác thực được tài khoản công ty, dẫn đến không thể lấy dữ liệu chi tiết CV.

Thứ hai, cần đồng bộ đúng giữa slug ở phía Frontend và id ở phía Backend. Trên Frontend, giá trị được lấy từ params.slug, còn ở Backend lại được nhận thông qua req.params.id. Nếu truyền sai hoặc route không khớp thì API sẽ không lấy được dữ liệu.

Thứ ba, ở Backend cần kiểm tra quyền truy cập dữ liệu. Không chỉ cần tìm CV theo id, mà còn phải kiểm tra công việc liên quan đến CV đó có thuộc về công ty đang đăng nhập hay không. Nếu bỏ qua bước này thì một công ty có thể truy cập vào CV ứng tuyển của công ty khác, gây mất an toàn dữ liệu.

Thứ tư, khi dữ liệu trả về không hợp lệ hoặc không tìm thấy CV/job tương ứng, chương trình cần xử lý cẩn thận để tránh lỗi khi truy cập vào các thuộc tính của giá trị null.

5. Hướng giải quyết

Để xử lý các khó khăn trên, em đã áp dụng một số giải pháp sau:

Ở phía Frontend, em sử dụng headers() của Next.js để lấy cookie từ request hiện tại, sau đó gửi cookie này kèm theo request gọi API Backend. Nhờ vậy, Backend có thể xác thực được tài khoản công ty đang đăng nhập.

Ở phần gọi API, em sử dụng đường dẫn:

```
/company/cv/detail/${slug}
```

Trong đó slug được lấy trực tiếp từ URL. Backend nhận giá trị này thông qua route:

```
/cv/detail/:id
```

Như vậy, slug ở Frontend chính là id ở Backend.

Ở phía Backend, em sử dụng middleware:

```
authMiddleware.verifyTokenCompany
```

Middleware này giúp kiểm tra token của tài khoản công ty trước khi cho phép truy cập vào controller xử lý chi tiết CV.

Trong controller detailCV, em kiểm tra CV có tồn tại hay không bằng cách tìm theo `_id`. Sau đó, tiếp tục tìm job tương ứng với điều kiện `companyId` phải trùng với công ty đang đăng nhập. Cách xử lý này giúp đảm bảo tính bảo mật dữ liệu.

Ngoài ra, em sử dụng `try...catch` để bắt các lỗi phát sinh trong quá trình truy vấn database hoặc khi dữ liệu không hợp lệ. Khi xảy ra lỗi, API sẽ trả về thông báo lỗi thay vì làm chương trình bị dừng đột ngột.

6. Kế hoạch tuần tiếp theo

Trong tuần tiếp theo, em dự kiến tiếp tục hoàn thiện các chức năng liên quan đến quản lý CV trong hệ thống. Cụ thể, em sẽ xây dựng tính năng **duyet và từ chối CV** để nhà tuyển dụng có thể xử lý hồ sơ ứng viên sau khi xem chi tiết. Khi nhà tuyển dụng chọn duyệt hoặc từ chối, hệ thống sẽ cập nhật lại trạng thái CV trong cơ sở dữ liệu.

Bên cạnh đó, em sẽ thực hiện chức năng **xóa CV** tại trang quản lý CV. Chức năng này giúp nhà tuyển dụng loại bỏ những CV không cần tiếp tục quản lý. Trước khi xóa, hệ thống cần có xác nhận để tránh thao tác nhầm.

Ngoài các chức năng phía nhà tuyển dụng, em cũng sẽ xây dựng trang **quản lý CV đã gửi** dành cho ứng viên. Trang này giúp ứng viên xem lại các CV đã ứng tuyển, thông tin công việc đã gửi và trạng thái xử lý của từng CV.

Sau khi hoàn thành các chức năng trên, em sẽ tiến hành kiểm thử lại toàn bộ luồng quản lý CV, bao gồm duyệt CV, từ chối CV, xóa CV và hiển thị danh sách CV đã gửi, nhằm đảm bảo hệ thống hoạt động ổn định và đúng yêu cầu.